SMART COURSE ENROLLMENT SYSTEM

Python Full Stack Summer Internship (IT3711)

Sindhuja.S – 810023205054
Department of Information Technology
University College of Engineering (BIT Campus), Anna University, Tiruchirappalli

Internship – Astonish InfoTech Pvt. Ltd.

- Software & web solutions company, Tiruchirappalli, Tamil Nadu
- Process: Discuss → Design → Develop → Deploy
- Services: web, software, mobile, ERP, CRM, domain & hosting
- Domains: AI/ML, IoT, Embedded, VLSI, Cloud, Cyber Security
- Duration: 01/06/2026 – 30/06/2026 (30 days)
- Project: Smart Course Enrollment System (Python Full Stack)

Objectives

- Build a web app with Python, FastAPI, Jinja2, HTML/CSS/JS and MySQL
- Design a relational database for categories, courses and enrollments
- Let learners browse categories and filter courses by category
- Provide an online enrollment form with input validation
- Login page and dashboard with summary and recent enrollments
- Reduce paperwork and data-entry errors; responsive UI; automatic DB setup

Existing System & Its Problems

Manual process

Paper forms re-typed into registers or spreadsheets

Scattered info

Instructor, duration, seats spread across brochures

Limited access

Office hours only, in person or by phone

No confirmation

No instant acknowledgement of a request

Hard to track

No easy count of applicants per course

Long queues

Waiting time during admission periods

Proposed System

- Centralized, web-based enrollment platform
- Category browsing and course catalog with filtering
- Online enrollment – stored with status "Pending"
- Instant success / error feedback
- Login page and staff dashboard
- Automatic table creation and sample data on first start
- Responsive design for desktop and mobile

Technology Stack

Frontend

HTML5, CSS3, JavaScript
Dark gradient, card layout

Backend

Python, FastAPI
Uvicorn server

Templating

Jinja2 server-side rendering

Database

MySQL
mysql-connector-python

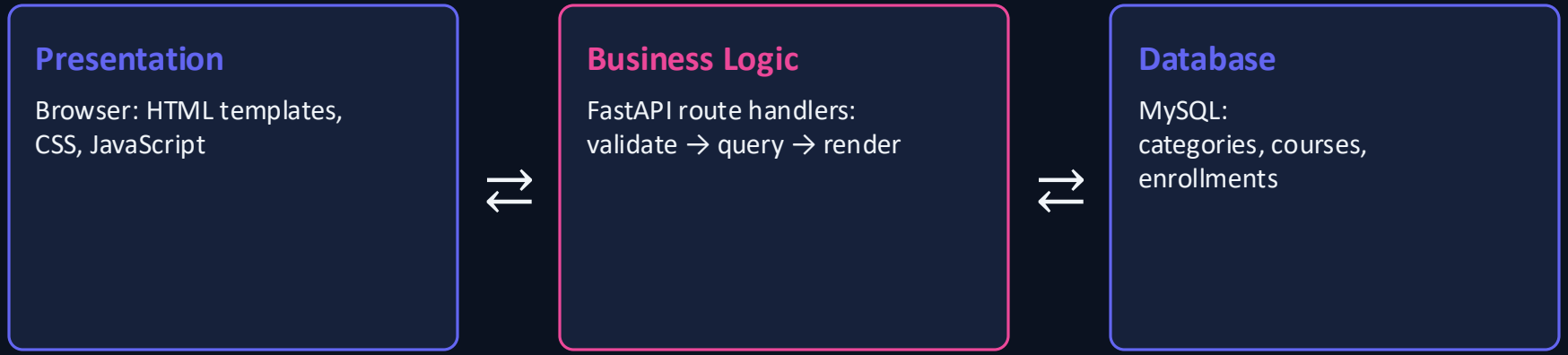
Tools

VS Code, MySQL Workbench
Virtual environment

Validation

HTML attributes + server-side checks
Pydantic model

Three-Tier Architecture



Request → route handler → SQL query → Jinja2 template → HTML response to the browser

System Modules

- Home & Categories
- Course Catalog & Filter
- Online Enrollment
- Login
- Dashboard
- Contact
- Database Mgmt
- Status & Notification

Each module is a FastAPI route plus a Jinja2 template; all share one stylesheet and a footer script.

Database Design

categories

id (PK), name (unique), description, image, color, created_at

courses

id (PK), title, category_id (FK), instructor, duration, level, description, seats

enrollments

id (PK), student_name, student_email, course_name, category_name, phone, status = Pending, created_at

One category → many courses (foreign key). init_db() also adds missing columns to older enrollments tables.

Algorithm – Enrollment & Login

Enrollment

1. Open form; load categories & courses
2. Browser checks required fields & email
3. POST to server, strip spaces
4. INSERT into enrollments (parameterised)
5. Success message – or log error and show it

Login

1. Enter username and password
2. POST to server, strip spaces
3. Empty field → error message
4. Password < 4 characters → error
5. Valid → HTTP 303 redirect to /dashboard

Code – Course Catalog Filter

```
@app.get("/courses", response_class=HTMLResponse)
async def courses_page(request: Request):
    selected_category = request.query_params.get("category")
    query = """
        SELECT c.*, cat.name AS category_name
        FROM courses c
        JOIN categories cat ON c.category_id = cat.id
    """
    params = ()
    if selected_category:
        query += " WHERE cat.name = %s"
        params = (selected_category,)   # parameterised
    query += " ORDER BY c.id"
```

Parameterised WHERE clause protects against SQL injection.

Code – Enrollment Route

```
@app.post("/enroll", response_class=HTMLResponse)
async def enroll_student(request: Request):
    try:
        form = await request.form()
        student_name = str(form.get("student_name","")).strip()
        ...
        cursor.execute("""
            INSERT INTO enrollments
            (student_name, student_email, course_name,
             category_name, phone)
            VALUES (%s, %s, %s, %s, %s)""", (...))
        return TemplateResponse(..., {"success": "Enrollment submitted..."})
    except Exception as exc:
        traceback.print_exc()
        return TemplateResponse(..., {"error": str(exc)})
```

Sample Output – Home & Login

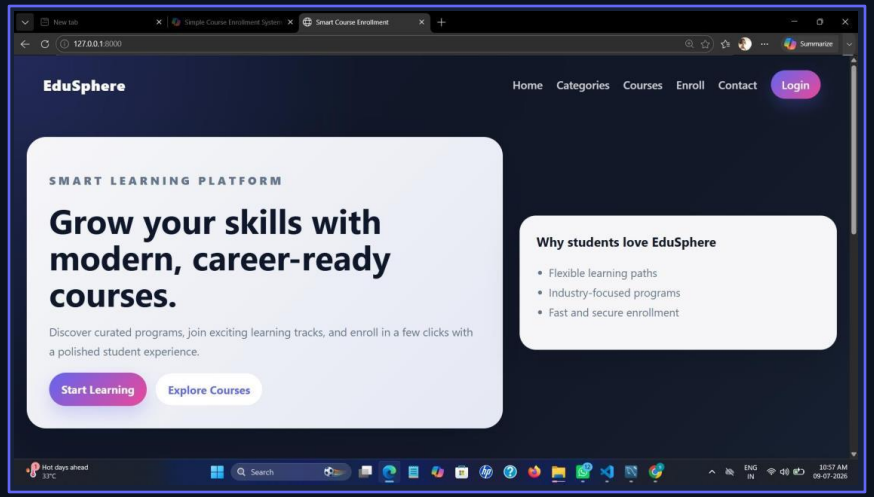


Figure 9.1: Home page

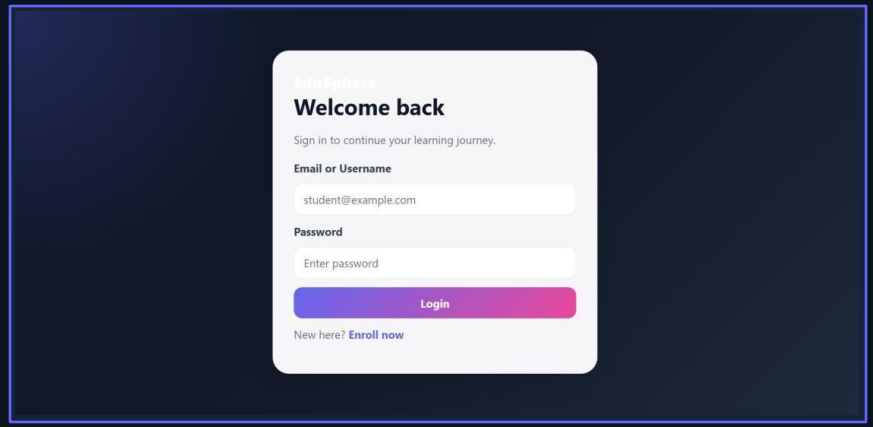


Figure 9.2: Login page

Sample Output – Categories & Course Catalog

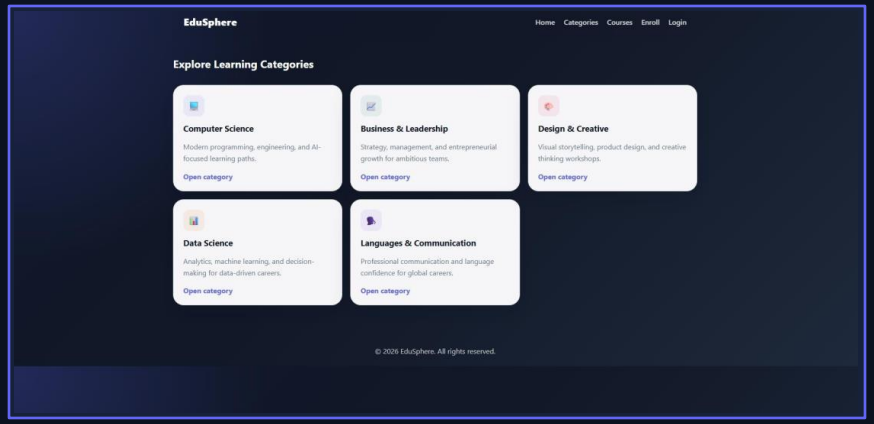


Figure 9.3: Categories page

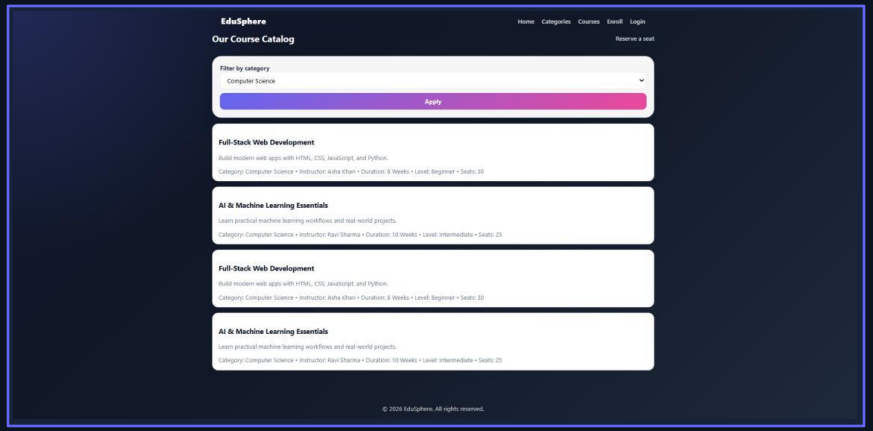


Figure 9.4: Catalog filtered by Computer Science

Sample Output – Enrollment

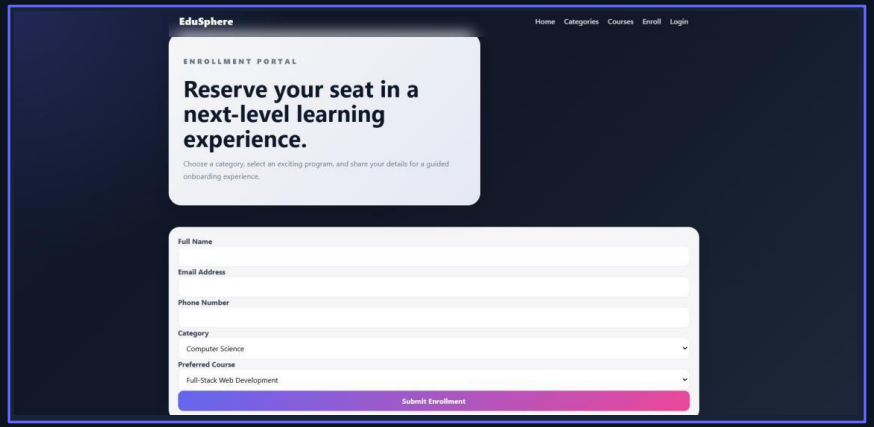


Figure 9.5: Enrollment form

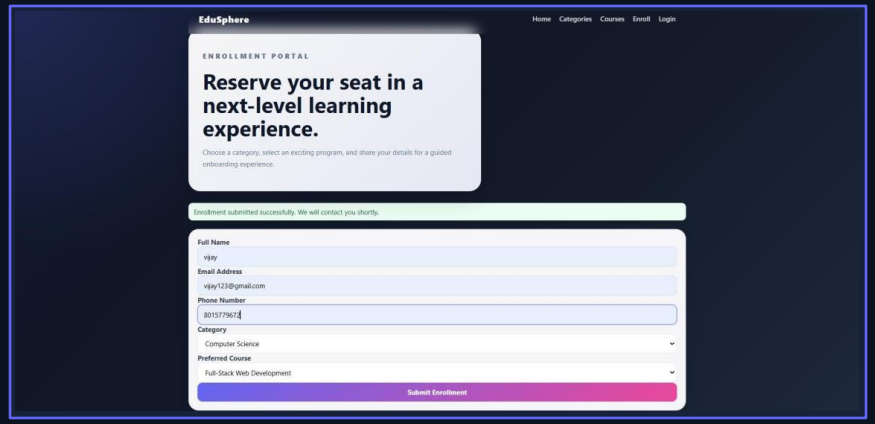


Figure 9.6: Enrollment submitted successfully

Testing & Analysis

- Module-wise functional testing of all pages and forms
- Edge cases: empty fields, invalid email, short password, DB not running
- Fast page loads – simple queries, small data set
- Responsive card layout adapts to screen sizes
- Compared with manual process: one place for info, single-form enrollment, fewer errors, instant dashboard view

Observed Limitations

Duplicate courses

Sample data loaded twice (SQL script + auto setup)

Login only validates input

No credential check against stored accounts

Dashboard count capped at 8

Based on latest-eight query

Category & course independent

Any course can pair with any category

Seats not reduced

Count shown but never decremented

Credentials in code

Move DB settings to environment variables

Future Work

- Secure authentication (hashing, JWT) with role-based access
- Administrator panel; approve / reject enrollments
- Seat management, waiting list, duplicate-enrollment prevention
- Dependent dropdowns; foreign keys in enrollments table
- Email/SMS notifications and payment gateway
- Search, pagination, analytics reports
- Pytest tests, Docker and cloud deployment, mobile app

Conclusion

- Digitizes and centralizes course enrollment, replacing paper-based registration
- Layered FastAPI + MySQL design with server-rendered, responsive pages
- All core modules implemented and tested successfully
- Gained practical experience in Python web development, SQL, templating, UI design and testing
- A solid foundation for a fuller enrollment management solution

Thank You

Questions?